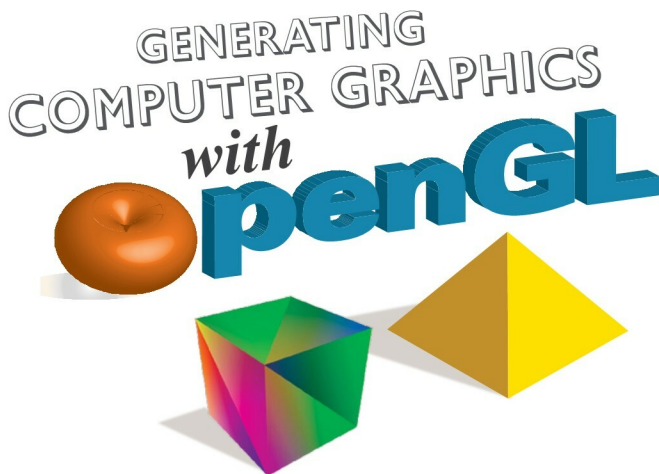




OpenGL is the industry's most widely used environment for the development of portable, interactive 2D and 3D graphics applications. The OpenGL API is open source and is platform independent. This article introduces OpenGL to our readers.

OpenGL was ported from the archaic Graphics Library (GL) system developed by Silicon Graphics Inc. as the means to program the company's high-performance specialised graphics workstations. GL was ported to OpenGL in 1992 so that the technology would be platform-independent, i.e., not just work on Silicon Graphics machines. OpenGL is a software interface to graphics hardware. It's the specification of an application programming interface (API) for computer graphics programming. The interface consists of different function calls, which may be used to draw complex 3D scenes. It is widely used in CAD, virtual reality, scientific and informational visualisation, flight simulation and video games.

The functioning of a standard graphics system is typically described by an abstraction called the graphics pipeline.

The graphics pipeline

The graphics pipeline is a black box that transforms a geometric model of a scene and produces a pixel based perspective drawing of the real world onto the screen. The graphics pipeline is the path that data takes from the CPU, through the GPU, and out to the screen. Figure 1 describes a minimalist workflow of a graphics pipeline.

Every pixel passes through at least the following phases: vertices generation, vertex shading, generating primitives, rasterisation, pixel shading, testing and mixing. After all these stages, the pixel is sent to a frame buffer, which later results in the display of the pixel on the screen. This sequence is not a strict rule of how OpenGL is implemented but provides a reliable guide for predicting what OpenGL will do.

A list of vertices is fed into the GPU that will be put together to form a mesh. Information present with the vertices need not be restricted to its position in 3D space. If required, surface normals, texture coordinates, colours, and spatial coordinate values generated from the control points might also be associated with the vertices and stored. All data, whether it describes geometry or pixels, can be saved in a display list for current or later use.

The vertices that are processed are now connected to form the mesh. The GPU chooses which vertices to connect per triangle, based on either the order of the incoming vertices or a set of indices that maps them together. From vertex data, the next stage is the generation of primitives, which converts the vertices into primitives. Vertex data gets transformed by matrix transformations. Spatial coordinates are projected from the 3D space map onto a screen. Lighting calculations as well